BỘ XÂY DỰNG TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
TP.HCM VIỆN ĐÀO TẠO CHẤT LƯỢNG CAO

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

Database Design Course Report

SMARTMOVERS DATABASE SYSTEM

Giảng viên hướng dẫn:	**Bùi Trọng Hiếu**
Hà Mạnh Trí	**09420601497**
Văn Tấn Dũng	**060206010128**
Nguyễn Thành Thi	**054206007675**
Phạm Lê Văn Bảo	**075206025346**
Đinh Hoàng Gia Bảo	**075206003404**

INTRODUCTION TO THE SUBJECT

The Database Design course provides essential knowledge and skills for organizing, constructing, and managing data within modern information systems. Its primary objective is to equip students with the ability to design databases that ensure data accuracy, consistency, integrity, and efficient performance.

The course covers key topics such as data requirements analysis, the development of Entity–Relationship Diagrams (ERD), data normalization techniques (including First, Second, and Third Normal Forms), relational schema design, and the use of Structured Query Language (SQL) for data definition and manipulation.

This course plays a vital role in the development of a wide range of software systems, including web applications, management systems, and enterprise-level information systems, where effective data organization and retrieval are critical to overall system performance and reliability.

ACKNOWLEDGEMENT

First and foremost, our team would like to express our sincere gratitude and deepest appreciation to Mr. Bui Trong Hieu, the lecturer of the Database Design course. We are truly thankful for his dedication, guidance, and continuous support throughout our learning process.

The professional knowledge, systematic thinking approach, and practical insights provided by the lecturer have greatly contributed to shaping our understanding of database design. His guidance has helped us develop logical thinking, a structured approach to problem-solving, and the ability to design database systems aligned with real-world requirements.

Through the lectures and detailed learning materials, we have gained a solid theoretical foundation, enabling us to independently study and gradually complete this project. The knowledge acquired not only supported us in solving technical challenges but also enhanced our awareness of the importance of database systems in modern information management.

Although we have made our best effort to complete this project with responsibility and seriousness, limitations in practical experience are inevitable. Therefore, we sincerely look forward to receiving further feedback and suggestions from the lecturer to improve our work and gain valuable experience for future academic and professional development.

Finally, we would like to wish Mr. Bui Trong Hieu good health, happiness, and continued success in his teaching and research career.

Table Of Contents

1. OVERVIEW OF THE TOPIC 3
1.1. Background and Problem 3
1.2. Objectives 3
1.3. Scope and Users 3
1.4. Significance 3
2. REQUIREMENTS ANALYSIS 3
2.1. Functional Requirements 3
2.2. Non-functional Requirements 4
2.3. Actors of SmartMovers System 5
3. DATA MODEL 5
3.1. Entity–Relationship Diagram (ERD) 5
3.1.1. Relationships, Cardinality and Participation 6
3.1.3. ERD Conclusion 8
3.2. DATABASE NORMALIZATION 9
3.2.1. First Normal Form (1NF) 9
3.2.2. Second Normal Form (2NF) 9
3.2.3. Third Normal Form (3NF) 10
3.2.4. Normalization Conclusion 10
4. DATABASE DESIGN 10
4.1. Database Schema 10
4.2. Table Definition 11
4.3. Constraints 13
5. PHYSICAL DESIGN 14
5.3. Data Storage Organization 15
5.4. Indexing Strategy 15
5.5. Implementing Constraints 16
5.6. Query Optimization 16
5.7. Data Integrity and Security 16
5.8. Scalability 17
6. TESTING – SMARTMOVERS DATABASE SYSTEM 17
6.1. Test Objectives 17
6.2. Test Scope 17
6.3. Test Results 20
6.4. Evaluation 21
6.5. Limitations 21
7. CONCLUDE 21

1. OVERVIEW OF THE TOPIC

1.1. Background and Problem

In the context of a rapidly developing transportation industry, managing customer data, orders, and inventory is becoming increasingly complex. Manual management methods or fragmented systems are prone to errors, duplication, and data inconsistencies. Therefore, building a database system within the field of Database Design is necessary to address these issues.

1.2. Objectives

The goals of the SmartMovers system are:

- Storing data accurately and consistently
- Supporting customer, goods, order, and warehouse management
- Optimizing data querying and processing
- Automating shipping cost calculations
- Supporting statistics and decision-making

1.3. Scope and Users

The project focuses on requirements analysis, data model design, and database construction with basic queries. The system serves users such as administrators and operations staff. Advanced functionalities such as user interface or practical implementation are outside the scope of the project.

1.4. Significance

The system helps improve data management efficiency, reduce operational errors, and support transportation businesses in making data-driven decisions. It also serves as a foundation for developing logistics management systems in practice.

2. REQUIREMENTS ANALYSIS

2.1. Functional Requirements

The SmartMovers system needs to support the following key functions:

1. **Customer Management**
   The system shall allow users to create, update, delete, and search customer information, including customer ID, name, address, phone number, and category.

2. **Product Management**
   The system shall manage product information, including product ID, product name, and risk type (No Risk or High Risk).
3. **Depot Management**
   The system shall store and manage depot information, including depot ID and location.
4. **Transport Unit Management**
   The system shall manage transport units, including vehicle (lorry), driver, assistant, and associated depot.
5. **Container Management**
   The system shall manage container types and assign containers to transport units.
6. **Job Management**
   The system shall allow users to create and manage transport jobs, including assigning customers, depots, start locations, and destinations.
7. **Load Management**
   The system shall manage loads within each job, including product type, transport unit assignment, and load size (small, medium, large).
8. **Cost Calculation**
   The system shall automatically calculate transportation costs based on:
   a. Customer category
   b. Product risk type
   c. Load size
   using predefined payment rules stored in the system.
9. **Data Entry Interface**
   The system shall provide user interfaces for entering and updating data for customers, products, jobs, and loads.
10. **Dashboard and Monitoring**
    The system shall display summary statistics such as total customers, jobs, loads, and transport units.
11. **Reporting and Statistics**
    The system shall generate reports including:
    a. Job cost summary
    b. Load pricing details
    c. Job assignment overview
    d. Transport unit allocation
    e. Depot workload summary
    f. Customer load summary
12. **Search and Query Support**
    The system shall allow users to search and filter data across database tables.

2.2. Non-functional Requirements

Non-Functional Requirements of SmartMovers System

1. **Performance**
   The system should return query results and reports within a short response time (typically under 2–3 seconds for normal operations).
2. **Usability**
   The system should provide a user-friendly interface with clear navigation, forms, and dashboards for easy interaction.

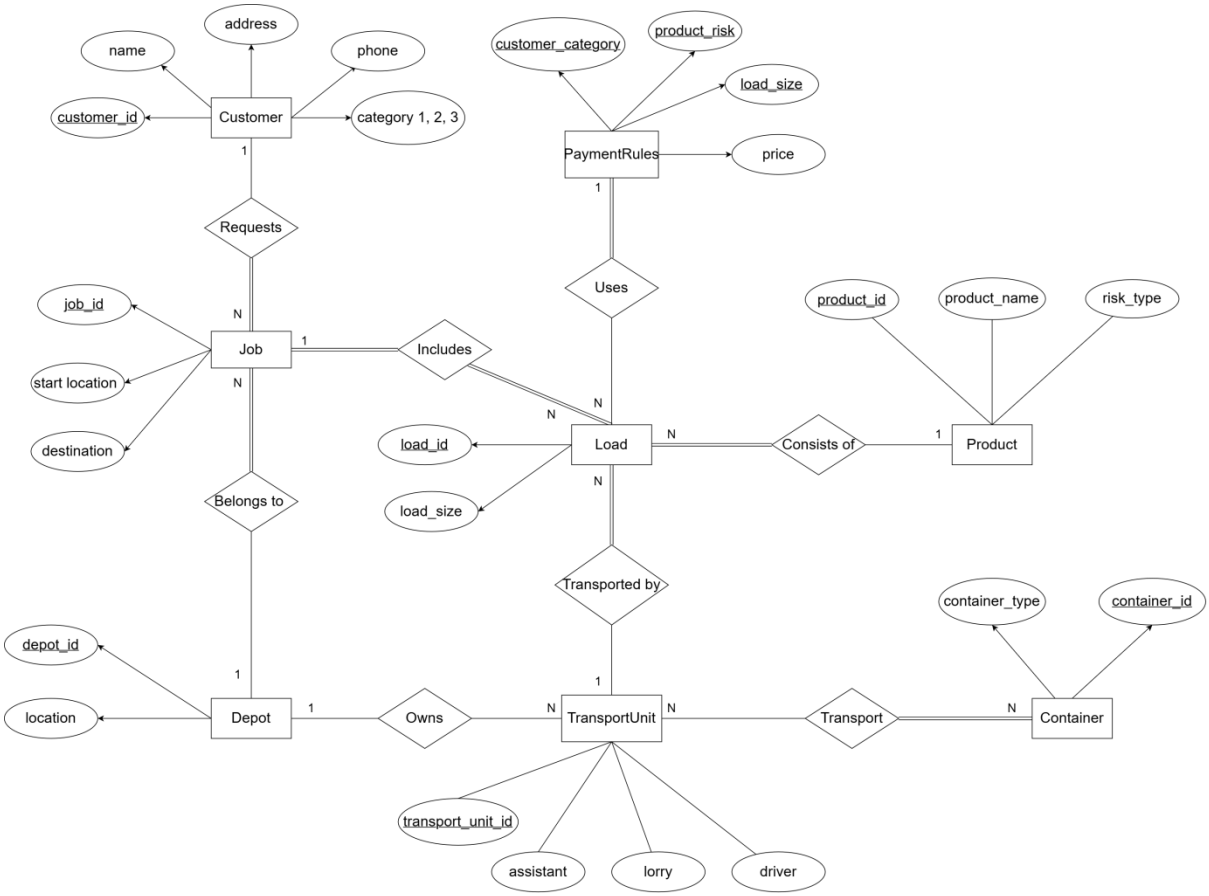
3. **Reliability**
   The system must ensure data consistency and integrity using primary keys, foreign keys, and constraints in the database.
4. **Scalability**
   The system should be able to handle an increasing number of customers, jobs, and loads without significant performance degradation.
5. **Security**
   The system should protect database access through authentication and prevent unauthorized data manipulation.
6. **Maintainability**
   The system should be modular, with separate components for database access and user interface, making it easier to maintain and extend.
7. **Data Integrity**
   The system must enforce valid data through constraints such as CHECK, NOT NULL, and foreign keys.
8. **Portability**
   The system should be deployable on different environments that support Python, Streamlit, and MySQL.
9. **Availability**
   The system should be available for use whenever required, especially for data entry and reporting operations.
10. **Accuracy**
    The system must ensure correct cost calculation based on defined business rules without manual intervention.

2.3. Actors of SmartMovers System

1. **Admin:** Manages all system operations including data entry, updates, and system monitoring.
2. **Staff (Operator):** Handles daily operations such as entering jobs, loads, and assigning transport units.
3. **Manager:** Views reports and analyzes system data for decision-making.
4. **System:** Automatically performs cost calculation based on predefined payment rules.

3. DATA MODEL

3.1. Entity–Relationship Diagram (ERD)



The system consists of the following main entities:

- Customer
- Job
- Depot
- Load
- Product
- TransportUnit
- Container
- PaymentRule

3.1.1. Relationships, Cardinality and Participation

Customer

- Primary Key: customer_id
- Attributes: name, phone, address, category (1, 2, 3)

Job

- Primary Key: job_id
- Foreign Keys: customer_id, depot_id
- Attributes: start_location, destination

Load

- Primary Key: load_id
- Foreign Keys: job_id, product_id, transport_unit_id
- Attribute: load_size

Product

- Primary Key: product_id
- Attributes: product_name, risk_type

Transport Unit

- Primary Key: transport_unit_id
- Foreign Key: depot_id
- Attributes: lorry, driver, assistant

Note: container_id is **not an attribute** of Transport Unit anymore.
Containers are linked through a many-to-many relationship (Transport).

Container

- Primary Key: container_id
- Attributes: container_type

Depot

- Primary Key: depot_id
- Attributes: location

PaymentRule

- Composite Primary Key: (customer_category, product_risk, load_size)
- Attribute: price

These attributes are used to determine the transport price in the system .

3.1.2. Relationships

Customer – Job

- One customer can have many job
- Each job belongs to exactly one customer

Job – Load

- One job can include multiple loads
- Each load belongs to exactly one job

Load – Product

- Each load is associated with exactly one product
- One product can be associated with multiple loads

Load – Transport Unit

- Each load is transported by one transport unit
- A transport unit can transport multiple loads over time

Depot – Job

- One depot manages multiple jobs

- Each job is assigned to one depot

Depot – Transport Unit

- One depot manages multiple transport units
- Each transport unit belongs to one depot

Transport Unit – Container

- Many transport units can use many containers
- Relationship is implemented through a junction table: Transport

This is a **many-to-many** (N–N) relationship, not a direct attribute.

PaymentRule – Load

- One payment rule applies to many loads
- Each load uses exactly one payment rule

3.1.3. ERD Conclusion

The ERD diagram fully models all entities, relationships, and cardinalities.
The relationships are clearly defined, including participation constraints (total or partial participation).
Many-to-many relationships are properly handled using junction (intermediate) tables.
The design is logical and consistent with the system requirements.

Cardinality and Participation Constraints

Relationship	Cardinality	Participation
Customer – Job	1 : N	Customer: Partial Job: **Total**
Depot – Job	1 : N	Depot: Partial Job: **Total**
Job – Load	1 : N	Job: Partial Load: **Total**
Product – Load	1 : N	Product: Partial Load: **Total**

TransportUnit – Load	1 : N	TransportUnit: Partial Load: **Total**
Depot – Transport Unit	1 : N	Depot: Partial TransportUnit: **Total**
TransportUnit – Container	N : N	TransportUnit: Partial Container: Partial
PaymentRule – Load	1 : N	PaymentRule: Partial Load: **Total**

3.2. DATABASE NORMALIZATION

3.2.1. First Normal Form (1NF)

The database satisfies First Normal Form (1NF) because:

- All attributes contain atomic (indivisible) values
- There are no repeating groups or multi-valued attributes

For example:

- The attributes **name** and **phone** in the **Customer** table each store a single value only
- The attribute **load_size** in the **Load** table does not contain a list of values

→ Therefore, the system satisfies 1NF.

3.2.2. Second Normal Form (2NF)

The database satisfies Second Normal Form (2NF) because:

- All tables have clearly defined primary keys.
- There are no partial dependencies on the primary key.

Tables such as Customer, Job, and Product use single-column primary keys, so partial dependency cannot occur in those tables.

The PaymentRule table uses a composite primary key (customer_category, product_risk, load_size), and the attribute price depends on the entire key.

The junction table Transport also uses a composite primary key (transport_unit_id, container_id), and it contains no non-key attributes that could have a partial dependency.

→ Therefore, the system satisfies 2NF.

3.2.3. Third Normal Form (3NF)

The database satisfies Third Normal Form (3NF) because:

- There are no transitive dependencies.
- All non-key attributes are directly dependent on the primary key.

Proof of no data redundancy:

- Customer information is stored only in the Customer table and is not duplicated in the Job table.
- Product information is stored only in the Product table and is not repeated in the Load table.
- Vehicle information is separated from the Depot table.

→ The data is properly decomposed, reducing redundancy.

Proof of no transitive dependency:

- In the PaymentRule table, the attribute price depends directly on the composite primary key (customer_category, product_risk, load_size).
- In the Job table, all attributes depend directly on job_id.
- No attribute depends on another non-key attribute (for example, destination in Job does not depend on start_location).

→ Therefore, there are no transitive dependencies, and the database satisfies 3NF.

3.2.4. Normalization Conclusion

The database has been normalized up to Third Normal Form (3NF).
There is no data redundancy and no transitive dependency.
The design ensures data integrity and minimizes update anomalies.

4. DATABASE DESIGN

4.1. Database Schema

The SmartMovers system is built with the following main tables:

1. **Customer:** stores customer information
2. **Depot**: stores regional depot information
   Product: stores product information
3. **Container:** stores container type
4. **TransportUnit**: transport vehicle
5. **Transport**: an intermediate table showing many-to-many relationships
6. **Job:** shipping order
7. **Load:** details of goods in each order
8. **PaymentRule:** rules for calculating costs

The tables are linked together using primary and foreign keys, ensuring that the data has a clear and consistent structure.

4.2. Table Definition

The tables are defined in SQL as follows:

```
CREATE TABLE Customer (
   customer_id VARCHAR(20) PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   address VARCHAR(255) NOT NULL,
   phone VARCHAR(20) NOT NULL,
   category VARCHAR(20) NOT NULL,
   CHECK (category IN ('Category 1', 'Category 2', 'Category 3'))
);

CREATE TABLE Depot (
   depot_id VARCHAR(20) PRIMARY KEY,
   location VARCHAR(100) NOT NULL
);

CREATE TABLE Product (
   product_id VARCHAR(20) PRIMARY KEY,
   product_name VARCHAR(100) NOT NULL,
   risk_type VARCHAR(20) NOT NULL,
   CHECK (risk_type IN ('No Risk', 'High Risk'))
);

CREATE TABLE Container (
   container_id VARCHAR(20) PRIMARY KEY,
   container_type VARCHAR(50) NOT NULL
```

```
);

CREATE TABLE TransportUnit (
   transport_unit_id VARCHAR(20) PRIMARY KEY,
   depot_id VARCHAR(20) NOT NULL,
   lorry VARCHAR(100) NOT NULL,
   driver VARCHAR(100) NOT NULL,
   assistant VARCHAR(100) NOT NULL,
   FOREIGN KEY (depot_id) REFERENCES Depot(depot_id)
);

CREATE TABLE Transport (
   transport_unit_id VARCHAR(20) NOT NULL,
   container_id VARCHAR(20) NOT NULL,
   PRIMARY KEY (transport_unit_id, container_id),
   FOREIGN KEY (transport_unit_id) REFERENCES TransportUnit(transport_unit_id),
   FOREIGN KEY (container_id) REFERENCES Container(container_id)
);

CREATE TABLE Job (
   job_id VARCHAR(20) PRIMARY KEY,
   customer_id VARCHAR(20) NOT NULL,
   depot_id VARCHAR(20) NOT NULL,
   start_location VARCHAR(100) NOT NULL,
   destination VARCHAR(100) NOT NULL,
   FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
   FOREIGN KEY (depot_id) REFERENCES Depot(depot_id)
);

CREATE TABLE `Load` (
   load_id VARCHAR(20) PRIMARY KEY,
   job_id VARCHAR(20) NOT NULL,
   product_id VARCHAR(20) NOT NULL,
   transport_unit_id VARCHAR(20) NOT NULL,
   load_size VARCHAR(20) NOT NULL,
   FOREIGN KEY (job_id) REFERENCES Job(job_id),
   FOREIGN KEY (product_id) REFERENCES Product(product_id),
   FOREIGN KEY (transport_unit_id) REFERENCES TransportUnit(transport_unit_id),
   CHECK (load_size IN ('small', 'medium', 'large'))
);
```

```
CREATE TABLE PaymentRule (
   customer_category VARCHAR(20) NOT NULL,
   product_risk VARCHAR(20) NOT NULL,
   load_size VARCHAR(20) NOT NULL,
   price DECIMAL(10,2) NOT NULL,
   PRIMARY KEY (customer_category, product_risk, load_size),
   CHECK (customer_category IN ('Category 1', 'Category 2', 'Category 3')),
   CHECK (product_risk IN ('No Risk', 'High Risk')),
   CHECK (load_size IN ('small', 'medium', 'large'))
);
```

4.3. Constraints

To ensure data accuracy, consistency, and integrity, the SmartMovers database applies several types of constraints, including Primary Keys, Foreign Keys, and CHECK constraints.Customer(customer_id)

- Depot(depot_id)
- Product(product_id)
- Container(container_id)
- TransportUnit(transport_unit_id)
- Job(job_id)
- Load(load_id)

Composite Primary Key:

- Transport(transport_unit_id, container_id)
- PaymentRule(customer_category, product_risk, load_size)

Foreign Key (FK)

Foreign keys are used to establish relationships between tables and enforce referential integrity. They ensure that values in one table must exist in the related table.Job.customer_id → Customer.customer_id

- Job.depot_id → Depot.depot_id
- Load.job_id → Job.job_id
- Load.product_id → Product.product_id
- Load.transport_unit_id → TransportUnit.transport_unit_id

- TransportUnit.depot_id → Depot.depot_id
- Transport linking TransportUnit and Container

The Transport table acts as a linking table to resolve the many-to-many relationship between TransportUnit and Container.

CHECK Constraints

CHECK constraints are used to restrict the range of valid values for specific attributes, ensuring data validity.

- category ∈ ('Category 1', 'Category 2', 'Category 3')
- risk_type ∈ ('No Risk', 'High Risk')
- load_size ∈ ('small', 'medium', 'large')

4.6. Evaluation

The database has been designed and implemented to ensure:

- Clear, well-structured data with minimal redundancy
- Tightly linked tables through foreign keys
- Good support for queries and statistics
- Scalability for the future

5. PHYSICAL DESIGN

Physical design is the step of specifically implementing the logical data model onto a database management system (DBMS), including selecting data types, storage structures, indexes, and optimizing performance in Database Design.

In the SmartMovers system, physical design is performed on the MySQL platform to ensure query performance, integrity, and scalability.

5.1 Database Management System and Deployment Environment

1. DBMS: MySQL
2. Storage Engine: InnoDB
3. Usage Environment: Localhost (XAMPP / MySQL Server)

The InnoDB storage engine is selected due to its strong support for:

- **Foreign Key constraints**, which ensure referential integrity between tables
- **ACID properties (Atomicity, Consistency, Isolation, Durability)**, which guarantee reliable transaction processing

InnoDB is particularly suitable for systems with frequent transactions, such as transportation and logistics management systems, where data consistency and reliability are critical.

5.2. Data Types Design

The selection of appropriate data types plays a crucial role in optimizing system performance and storage efficiency.

The design follows these principles:

- **Use VARCHAR for variable-length data** such as names, addresses, and identifiers to optimize storage usage
- **Choose appropriate data lengths** to avoid unnecessary memory consumption while still accommodating expected data values
- **Use DECIMAL for financial data** (e.g., price) to ensure precision and avoid rounding errors
- **Apply NOT NULL constraints** where necessary to ensure mandatory data is always provided
- **Use CHECK constraints** to restrict values to predefined domains (e.g., category, risk type, load size)

These design choices help improve database efficiency, maintain data accuracy, and support scalable system performance.

This design helps to: save memory, increase query speed, and facilitate scalability.

5.3. Data Storage Organization

Data is stored in separate copies:

- Customer, Product → Static data
- Job, Load → Transaction data
- PaymentRule → Business logic data

This separation helps: Easier maintenance, reduced redundancy, and faster query speed

5.4. Indexing Strategy

Indexes are used to improve query performance, especially for search and JOIN operations.

- **Primary Index:**
  Automatically created on primary key columns such as:

  a. customer_id
  b. job_id
  c. product_id
  d. load_id
  These indexes enable fast data retrieval by unique identifiers.

- **Foreign Key Index:**
  Applied to frequently joined columns, including:

  a. Job.customer_id
  b. Job.depot_id
  c. Load.job_id
  d. Load.product_id
  e. Load.transport_unit_id

  These indexes help optimize JOIN operations between related tables.

5.5. Implementing Constraints

Constraints are applied directly at the physical level.

- **Primary Key**

Ensure each record is unique.

- **Foreign Key**

Ensure the links between the tables are correct.

- **CHECK Constraint**

CHECK (load_size IN ('small','medium','large'))

- **NOT NULL**

Ensures that required data is provided.
Meaning: Prevents missing or invalid data and helps maintain system integrity.

5.6. Query Optimization

The system applies basic query optimization techniques:

- Using indexes on key columns to improve **JOIN** performance
- Avoiding unnecessary JOIN operations
- Using **GROUP BY** efficiently on relevant columns
- Creating **VIEWS** to simplify complex queries and improve reusability

These techniques help enhance query execution speed and maintain system efficiency.

5.7. Data Integrity and Security

a. Use FK to ensure valid associations
b. CHECK to validate input data
c. Expandable permissions:
d. Admin (full access)

e. User (limited read/write)

5.8. Scalability

The system can be scaled by:

- Adding indexes to large tables
- Partitioning data over time
- Optimizing queries
- Connecting to backend APIs

6. TESTING – SMARTMOVERS DATABASE SYSTEM

This section describes the system testing process to ensure that the database and related functions operate correctly, stably, and meet the requirements analyzed in the Database Design.

6.1. Test Objectives

- Ensure functionalities work correctly
- Verify data accuracy
- Verify constraints (PK, FK, CHECK)
- Evaluate error handling capabilities and system stability

6.2. Test Scope

The system was tested based on 18 test cases (TC-01 → TC-18)

Functional Testing:

Check the main functions:

- Dashboard
- Data Entry (Customer, Product, Job, Load)
- Reports (Job Cost, Depot Workload, …)

TC - ID	Function	Test steps	Expected result
TC-01	Summary Metrics	1. Open the application 2. Navigate to the Dashboard menu.	Metrics (Customers, Depots, Jobs, etc.) display the correct total counts based on the database records.

TC-02	Job Cost Overview	1. View the "Job Cost Overview" table on the Dashboard.	Displays Job ID, Customer Name, and Total Cost correctly as retrieved from the JobCost view.
TC-03	Valid Customer Entry	1. Go to Data Entry -> Customer tab. 2. Enter ID: C999, Name: Logistics Ltd, Address: NY, Phone: 12345, Category: Category 1. 3. Click "Save Customer".	"Customer saved successfully" message appears. The new customer appears in the "Customer List" table below.
TC-04	Mandatory Field Validation	1. Leave ID or Name empty. 2. Click "Save Customer".	Warning message: "Customer ID and Customer Name are required." No record is added to the database.
TC-05	Duplicate Primary Key	1. Enter a Customer ID that already exists in the DB. 2. Click "Save Customer".	Database error message (Duplicate entry) is displayed. The record is not saved.
TC-06	Valid Product Entry	1. Go to Product tab. 2. Enter ID: P500, Name: Chemical X, Risk: High Risk. Click "Save Product".	Product is saved successfully and populates in the Product List.
TC-07	Job Entry with Dropdowns	1. Go to Job tab. 2. Select a Customer and Depot from the dropdown lists. 3. Enter Locations and click "Save Job".	The system correctly maps the label (Name) to the ID. Job is created and linked correctly to the selected Customer/Depot.
TC-08	Load Assignment	1. Go to Load tab. 2. Select a Job ID, Product, and Transport Unit. 3. Select Load Size	Load is successfully linked to the Job. Pricing logic (checked in Reports) should now apply to this load.

		(Small/Medium/Large) and Save.	
TC-09	Job Cost Summary	1. Select "Job Cost Summary" from the Report Type dropdown.	Displays the total transport cost per Job aggregated from the JobCost view.
TC-10	Load Pricing Logic	1. Select "Load Pricing Details".	Verifies that prices are correctly joined from PaymentRule based on Customer Category, Product Risk, and Load Size.
TC-11	Depot Workload	1. Select "Depot Workload Summary".	Shows a count of jobs per depot. Depots with zero jobs should show '0' if a LEFT JOIN is used.
TC-12	Transport Unit Allocation	1. Select "Transport Unit Allocation".	Correctly lists Lorry details, Drivers, and the specific Containers assigned to them.

UI & Interaction Testing

Test:

- Display data tables
- Search
- User interaction

TC-13	Table Switching	1. Navigate to Database Tables. 2. Select different tables from the dropdown (e.g., TransportUnit, Load).	The data grid updates instantly to reflect the selected table's schema and data.
TC-14	Keyword Search	1. Select the Customer table. 2. Type a partial name or ID in the "Search" box.	The dataframe filters rows in real-time, showing only matches (case-insensitive).
TC-15	Empty Search Results	1. Type a non-existent string	The table displays as empty but the application does not crash.

System & Reliability Testing

Check:

- Database connection errors

- UI status
- Real-time data updates

TC-16	DB Connection Failure	1. Temporarily stop the MySQL service. 2. Refresh the Streamlit app.	The app displays a "Database error" message instead of crashing with a traceback.
TC-17	UI State Persistence	1. Enter data into a form but don't submit. 2. Switch sidebar menus and come back.	Data in unsubmitted forms might reset; verify if this is acceptable or if ***st.session_state*** is required.
TC-18	Data Rerun	1. Add a new record in Data Entry. 2. Check the "Database Tables" menu immediately.	The new record must appear immediately without needing a manual app restart.

6.3. Test Results

System Functionality

All CRUD functions work correctly

Data is stored and retrieved correctly

Data Constraints

- Primary Key prevents duplicate data
- Foreign Key ensures valid links
- CHECK and NOT NULL prevent incorrect data

Data Query and Processing

- JOIN works correctly between tables
- GROUP BY provides correct statistical results
- VIEW (JobCost) returns accurate data

User Interface

- Data is displayed correctly and updated quickly
- Search works in real-time
- No errors occur when there is no data

System Stability

- No crashes when database connection is lost
- Clear error handling (error message)
- New data is updated immediately (rerun)

6.4. Evaluation

Based on test results:

- The system fully meets functional requirements
- Data is accurate and consistent
- User-friendly and easy-to-use interface
- The system is stable in error situations

6.5. Limitations

- Performance testing has not been performed on large datasets.
- Security testing has not been conducted.
- Some UI states may reset when switching pages.

6.6 Development Direction

- Add performance testing (stress testing)
- Improve UI/UX
- Integrate logging and monitoring
- Deploy the system in practice (web/app)

7. CONCLUDE

Within the scope of this project, the SmartMovers database system was designed and fully implemented based on the principles of Database Design. The implementation process included requirements analysis, data model building, normalization, logical and physical design, to query implementation and system testing.

First, the system clearly defined key entities such as Customer, Job, Product, Load, Depot, and the relationships between them through an ERD diagram. Data was organized logically and normalized to Third Normal Form (3NF), eliminating redundancy and ensuring consistency.

Next, the implementation of the database on MySQL with constraints such as Primary Key, Foreign Key, and CHECK ensured data integrity. Queries using JOIN, GROUP BY, and VIEW allowed the system to process data efficiently, especially in calculating shipping costs and supporting reporting.

Furthermore, the system underwent comprehensive testing through 18 test cases, covering key functions, user interface, and error scenarios. The test results showed that the system operates stably, accurately, and fully meets the stated requirements.

However, the system still has some limitations, such as not being optimized for large datasets and lacking full integration of practical functions like advanced user interfaces or real-time shipping tracking. This will be a direction for future development.

In summary, the SmartMovers system has achieved its goal of building a complete, well-structured, efficient, and scalable database. The project not only reinforced theoretical knowledge but also provided practical experience in designing and implementing database systems.